# C compliant coding
# Training Material

Prepared By: R. Udayalakshmi.
Prepared on: 23rd December 2002.

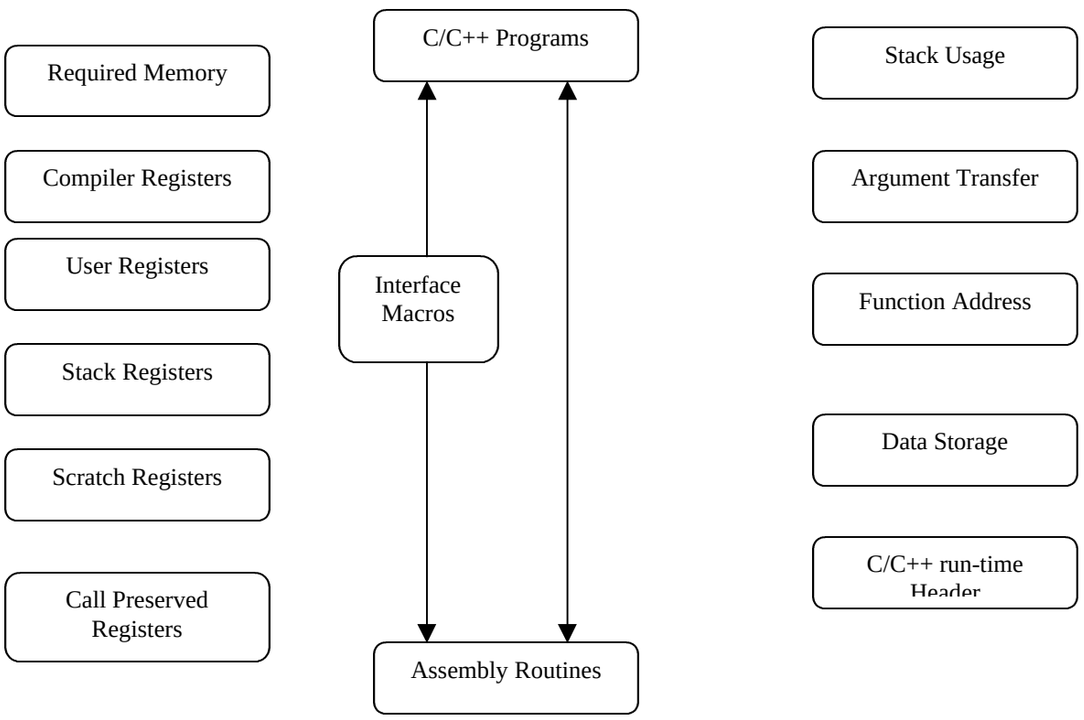
**Introduction:**

This training material is intended for programmers in ADSP SHARC family processors (viz) ADSP 21065L, 21161 etc. This gives a brief overview of the C compliance coding.

**Why C?**

In the early days of computing, programs were written in **machine language**: a series of 0s and 1s, and were not easily comprehensible or portable. Programming in machine language is very tedious because the instructions are primitive and often require detailed knowledge of how the computer works. The next development was the creation of **assembly language**, where instructions and memory locations were referred to by a mnemonic form. However, the widespread use of processors in problems solving became more easier only after the advent of **high level programming languages**. In order to run a high level program on a processor it must be converted to machine code by another program known as a **compiler**. High level programming languages are an attempt to alleviate the problems of programming in machine and assembly language and provide a level of machine independence.

**C/ C++ Run-Time Model:**




The above figure illustrates the ADSP-21xxx run-time environment model. The C/C++ run-time environment is a set of conventions that C and C++ programs follow to run on ADSP-21xxx DSPs. Assembly routines that we use link to C or C++ must follow these conventions.

**Memory Usage:**

The cc21k run-time environment requires that a specific set of memory section names are used for placing code and data in memory.

Seg_pmco : Program Memory Code storage
Seg_dmda: Data Memory Data Storage
Seg_pmda: Program Memory Data Storage
Seg_stak: Run-time Stack Storage
Seg_Heap: Run-time Heap Storage
Seg_Init: Initialization Data Storage
Seg_rth:Run-time Header Storage (Interrupt vector table).

**Compiler Registers:**

The cc21k C/C++ run-time environment reserves a set of registers for its own use. The user should not modify these registers violating the values as listed below.

M5 = M13 = 0.
M6 = M14 = 1.
M7 = M15 = -1.
B6 & B7 are used as stack base. Do not modify.
L6 & L7 are used as stack length. Do not modify.

L0 = l1 = l2 = l3 = l4 = l5 = l8 = l9 = l10 = l11 = l12 = l13 = l14 = l15 = 0. Modify these L registers for temporary use and restore when done.

**User Registers:**

The user can reserve registers for his inline assembly code or assembly language routines. C run-time library does not use these registers.
I0,b0,l0,m0,i1,b1,l1,m1,i8,b8,l8,m8,i9,b9,l9,m9, mrb, ustat1, ustat2.

**Call Preserved Registers:**

The cc21k C/C++ run-time environment specifies a set of registers whose contents must be saved and restored. Assembly function must save these registers during the function's prologue and restore the registers as part of the function's epilogue. If a function does not change a particular register, it does not need to save and restore the register.

B0,b1,b2,b3,b5,b8,b9,b10,b11,b14,b15, i0,i1,i2,i3,i5,i8,i9,i10,i11,i14,i15,mode1, mode2, mrb, mrf, m0,m1,m2, m3, m8, m9, m10, m11, r3, r5, r6, r7, r9, r10, r11, r13, r14, r15, ustat1 and ustat2 are the list of those registers.

**Scratch registers:**
The cc21k C/C++ run-time environment specifies a set of registers whose contents do not need to be saved and restored.

b4,b12,b13,r0,r1,r2,r4,r8,r12,i4,i12,m4,m12,i13.

**Stack Registers:**

The cc21k C/C++ run-time environment reserves a set of registers for controlling the run-time stack. These registers may be modified for stack management, but must be saved and restored.

I7 – Stack pointer
I6 – Frame pointer
I12 – Return address

**Alternate Registers:**

The C/C++ run-time environment model does not use any of the alternate registers because these registers are available for use in assembly language only. The super fast interrupt dispatcher uses context switching rather than saving registers on run-time stack.

**Arguments Transfer:**

The C/C++ run-time environment uses a set of registers and the run-time stack to transfer function parameters to assembly routines. The run-time environment attempts to pass the first three parameters in a function call using registers and passes any remaining parameters on the run-time stack. The convention is to pass the function's first parameter in r4, the second parameter in r8, and the third parameter in r12. If the function is declared to take a variable number of arguments then the last named parameter and any subsequent parameters are passes on the stack.

Eg. Pass(int a, float b, char c, float d);

The first three arguments a, b and c are passed in registers r4, r8 and r12 respectively. The fourth argument, d is passed on the stack.

**Run-Time Header Usage:**

The run-time header is an assembly language procedure that initializes the processor and sets up processor features to support the C/C++ run-time environment. The source code for the default run-time header is in the 060_hdr.asm file. The run-time header performs the following operations:

1. Initializes the C/C++ run-time environment.
2. Sets up the interrupt table.
3. Calls your main() routine.

**C/C++ and Assembly Interface:**

**a) Calling Assembly Language subroutines from C/C++ Programs:**
Eg.

C routine:

```
main()
    {
        add(a,b);
    }
```

Assembly routine:

```
    _add:
        entry;
        r0 = r4 + r8;
        exit;
```

b) **Calling C/C++ Functions from Assembly Language Programs**:

Eg.
Assembly Routine:

```
r4 = 10;
r8 = 12;
ccall(_Myfunc);
dm(result) =r0;
```

C routine:

```
int Myfunc(int n, int m)
{

int z;
     z = m + n;
     return z;
}
```

**Using Mixed C/C++ and Assembly Support Macros:**

a) entry
b) exit
c) leaf_entry
d) leaf_exit
e) ccall(x)
f) reads(x)
g) puts
h) gets(x)
i) alter(x)
j) save_reg
k) restore_reg

**Other things to Know:**

1. Naming conventions
2. Interrupt functions in C run-time model.
3. SIMD usage in C codes.
4. Inline Assembly coding.

**********